



Trabalho de Projeto

3ª Fase

1 Descrição Geral

A componente teórico-prática da disciplina de sistemas distribuídos consiste no desenvolvimento de um projeto, dividido em 4 fases, utilizando a linguagem de programação C [1], sendo que a realização de cada uma das fases é necessária para a realização da fase seguinte. Por essa razão, **é muito importante que consigam ir cumprindo os objetivos de cada fase, de forma a não hipotecar as fases seguintes.**

O projeto consiste em implementar um sistema de gestão de inventário para um conjunto de um stand automóvel, utilizando uma lista encadeada [2] para armazenar a informação dos carros.

Na fase 1 foram definidas estruturas de dados e implementadas várias funções para lidar com a manipulação dos dados que vão ser armazenados na lista. Também foi construído um módulo para a serialização de dados, que teve como objetivo familiarizar os alunos com a necessidade de serializar os dados para a comunicação em aplicações distribuídas.

A fase 2 teve como objetivo implementar um sistema cliente-servidor simples, sendo o servidor responsável por manter uma lista (usando os módulos construídos no projeto 1) e sendo o cliente responsável por comunicar com o servidor para realizar operações na lista. Nesta fase, em vez de se utilizar o módulo de serialização desenvolvido na fase 1 (que teria de ser consideravelmente estendido com outras funções), foram utilizados os **Protocol Buffers** da Google [3], que permitem automatizar a criação das funções para **serializar** (codificar) e **de-serializar** (descodificar) os dados a serem enviados/recebidos, a partir de um ficheiro com a descrição destes dados.

Na 3ª fase do projeto iremos criar um sistema concorrente que aceita e processa pedidos de múltiplos clientes em simultâneo através do uso de múltiplas *threads*. Mais concretamente, vai ser preciso:

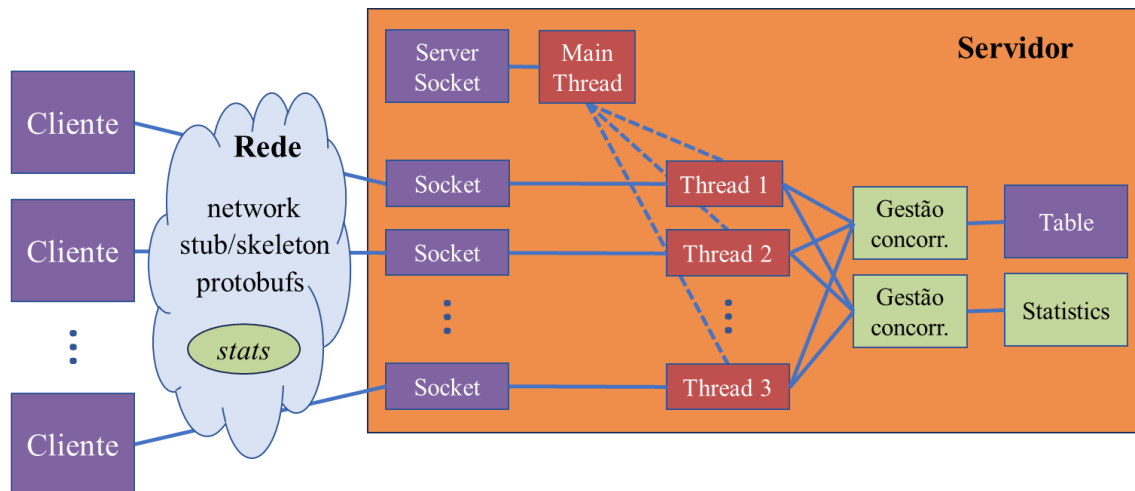
- Adaptar o servidor para este suportar múltiplos clientes ligados em simultâneo. Serão usadas *threads* para atender os clientes, seguindo um modelo *one-thread-per-client*. Neste sentido, é necessário adaptar o servidor para ter as seguintes *threads*:
 - A *thread* principal do programa, que fica responsável por aceitar novas ligações de clientes e por criar uma *thread* para atender cada novo cliente;
 - *Threads* secundárias, lançadas pela *thread* principal, que se vão manter ativas para atender os pedidos de um cliente até que este encerre a ligação.
- Implementar no servidor a criação de um ficheiro de log denominado `server.log`, onde são registados todos os eventos relevantes sobre o funcionamento do servidor, nomeadamente:
 - O momento (timestamp) em que um cliente estabelece uma ligação com o servidor;
 - O momento (timestamp) em que um cliente encerra a ligação;
 - O momento (timestamp) e o pedido que cada cliente envia ao servidor.
- Realizar a gestão da concorrência no acesso aos recursos partilhados do servidor, utilizando mecanismos de sincronização, nomeadamente *mutexes* e/ou variáveis de condições. Em particular, é necessário garantir:
 - Que não existam escritas concorrentes no ficheiro de log (`server.log`), assegurando que apenas uma *thread* de cada vez possa registar eventos;
 - Que o número máximo de clientes ligados simultaneamente não ultrapasse cinco, ou seja, se este limite for atingido, novas ligações devem ser imediatamente recusadas;
 - Que o acesso à lista encadeada (estrutura de dados partilhada) seja devidamente protegido através de exclusão mútua, evitando acessos concorrentes que possam comprometer a consistência dos dados.

Como nas fases anteriores, espera-se uma **grande fiabilidade** do servidor e do cliente, sendo para tal necessário tratar todas as condições de erro e garantir uma **correta gestão da memória**, evitando *memory leaks* ou acesso a zonas de memória inválidas. É importante notar que em sistemas cliente-servidor os servidores ficam em funcionamento permanente durante muito tempo (dias, meses ou até anos) e não é suposto que parem (*crash*) por causa de erros ou fecho de ligação dos clientes.

2 Descrição Específica

O objetivo específico da 3ª fase do Projeto é desenvolver uma aplicação do tipo cliente-servidor capaz de suportar múltiplos clientes de forma síncrona. Para tal, para além de aproveitarem o código desenvolvido nas fases 1 e 2 do projeto, os alunos devem fazer uso das APIs para programação com *threads*, *mutexes* e/ou *variáveis de condições*. A utilização de *threads* permitirá o atendimento de pedidos de clientes feitos de forma concorrente, e os *mutexes* e/ou

as *variáveis de condições* permitirão gerir a concorrência das *threads* no acesso às estruturas de dados partilhadas. A figura abaixo ilustra o novo modelo de comunicação que será usado na 3ª fase do projeto. Atenção que este modelo abstrai os detalhes de comunicação implementados na 2ª fase do projeto, colocando os módulos *network*, *stub* e *skeleton* na rede de interligação. A cor roxa representa o que já foi feito nas fases 1 e 2 do projeto, e as cores verde e vermelho representam o que é preciso fazer nesta 3ª fase do projeto.



2.1 Servidor *multi-threaded* com modelo *one-thread-per-client*

Para ser possível atender a vários clientes em paralelo, serão utilizadas *threads* do lado do servidor. O atendimento dos clientes deverá seguir um modelo *one-thread-per-client*, ou seja, de cada vez que um cliente se liga ao servidor, este deverá criar uma *thread* que ficará responsável por atender todos os pedidos deste cliente de forma exclusiva até este fechar a ligação, ou até ocorrer algum erro. A criação de *threads* é feita através da API do UNIX (*pthread*s).

Antes de aceitar uma nova ligação, o servidor deve **verificar o número atual de clientes ligados**.

Se o número de clientes ativos for **inferior a 5**, o servidor aceita a nova ligação e cria uma nova *thread* para o atender. Caso o número de clientes ativos seja igual ou superior a 5, o servidor deve enviar uma **mensagem adequada**, conforme explicado abaixo, e **rejeitar/fechar a ligação sem criar uma nova thread**.

Assim, apresenta-se de seguida a ideia geral do novo código a ser desenvolvido na função `network_main_loop(int listening_socket, struct list_t *list)` do módulo `network_server.c` do servidor, bem como do código a ser executado pelas *threads* secundárias. Assume-se que a *socket* de escuta (`listening_socket`) recebida como argumento já foi preparada para a receção de pedidos de múltiplas ligações num determinado porto (determinado pelo método `listen`). Cabe aos alunos traduzir esta ideia geral para o seu próprio código C (ou inventar outro algoritmo).

```
// Esboço do algoritmo a ser implementado na função network_main_loop
while (1) {
    connsockfd = accept(listening_socket);
    if (num_clientes_ativos < 5) {
        // cria uma nova thread passando-lhe connsockfd;
        // regista no log do servidor a ligação criada
        // incrementa o numero de clientes_ativos;
    } else {
        // não cria nova thread e rejeita a ligação;
        close(connsockfd);
    }
}

// Thread secundária de atendimento do cliente
while((message = network_receive(connsockfd)) != NULL) { // cliente não fechou conexão
    invoke(message); // Executa pedido contido em message
    network_send(connsockfd, message); // Envia resposta contida em message
}

close(connsockfd); // cliente fechou conexão ou ocorreu um erro
// regista no log do servidor o fecho da ligação com o cliente
// decrementa o num_clientes_ativos;
// termina a thread secundária;
```

Para implementar este comportamento, os alunos devem acrescentar os seguintes códigos de operação ao ficheiro

`sdmessage.proto:OP_BUSY = 100; OP_READY = 101`. Estes códigos serão utilizados para indicar o estado do servidor no momento da ligação. Quando o servidor não pode atender um novo cliente (porque já existem 5 clientes ativos), deverá enviar uma mensagem com o campo `opcode` definido para `OP_BUSY`. Caso contrário, o servidor enviará uma mensagem com o campo `opcode` definido para `OP_READY`, indicando que está disponível para servir o cliente. Do lado do cliente, na função `network_connect()`, o cliente deverá aguardar a receção desta mensagem de estado imediatamente após o `connect()`. Com base na mensagem recebida, o cliente pode então decidir se deve continuar a interação com o servidor ou terminar a ligação. Em particular, se a mensagem recebida tiver o código `OP_BUSY`, o cliente deverá apresentar a seguinte mensagem no terminal: **Server busy. Try again later.**

De notar que o atendimento de cada pedido será feito de forma síncrona, tal como já acontecia na 2ª fase do projeto. Isto significa que quando um cliente faz um pedido, ficará bloqueado à espera da respetiva resposta.

2.2 Estatísticas do Servidor

Nesta fase do projeto, o servidor terá de manter algumas informações estatísticas sobre o seu funcionamento. O servidor deverá registar no ficheiro `server.log` todas as informações relevantes sobre o seu funcionamento. Em particular, deverão ser registados:

- O momento em que um cliente estabelece uma ligação com o servidor;
- O momento em que um cliente encerra a ligação;
- O momento e o pedido que cada cliente envia ao servidor.

Cada linha do ficheiro `server.log` deve seguir um formato bem definido:

`<timestamp> <client_address>:<port> <event_type> <operation> [<content_type>] [<argument>]`

- `<timestamp>` – O timestamp de cada evento registado no ficheiro `server.log` deve ser obtido através da função `gettimeofday()`, que permite recolher o tempo com resolução de microssegundos. Esta função deve ser chamada imediatamente antes de cada operação relevante (por exemplo, ao estabelecer ou encerrar uma ligação, ou ao receber um pedido de um cliente).
- `<event_type>` – `CONNECT`, `REQUEST`, ou `CLOSE`.
- `<operation>` – `OP_ADD`, `OP_GET`, etc.
- `<content_type>` – `CT_DATA`, `CT_MODEL`, etc.
- `<argument>` – valor associado ao pedido do cliente.

Por exemplo, o ficheiro `server.log` poderá conter entradas como as seguintes:

```
1761053956 127.0.0.1:7654 CONNECT
1761053972 127.0.0.1:7654 REQUEST OP_ADD CT_DATA Toyota Corolla 2021
1761053983 127.0.0.1:7654 REQUEST OP_GET CT_MODEL Corolla
1761053995 127.0.0.1:7668 CONNECT
1761054012 127.0.0.1:7654 REQUEST OP_SIZE CT_NONE
1761054015 127.0.0.1:7654 CLOSE
1761054021 127.0.0.1:7668 REQUEST OP_SIZE CT_NONE
1761054026 127.0.0.1:7668 CLOSE
```

Deverá ser possível executar comandos no servidor para analisar o ficheiro de log e verificar se os registos estão a ser efetuados corretamente, tal como exemplificado nos *comandos bash* a seguir. Note que estes comandos apenas servem de exemplo de utilidade dos logs dos servidores e não é necessário submeter nada relativo aos mesmos na entrega do projeto.

```
$ cat server.log
1761053956 127.0.0.1:7654 CONNECT
1761053972 127.0.0.1:7654 REQUEST OP_ADD CT_DATA Toyota Corolla 2021
1761053983 127.0.0.1:7654 REQUEST OP_GET CT_MODEL Corolla
1761053995 127.0.0.1:7668 CONNECT
1761054012 127.0.0.1:7654 REQUEST OP_SIZE CT_NONE
1761054015 127.0.0.1:7654 CLOSE
1761054021 127.0.0.1:7668 REQUEST OP_SIZE CT_NONE
1761054026 127.0.0.1:7668 CLOSE
# calcular o número de clientes que se conectaram ao servidor:
```

```

$ cat server.log | grep CONNECT | wc -l
2

# calcular o número de clientes que encerraram a sua ligação ao servidor:
$ cat server.log | grep CLOSE | wc -l
2

# calcular o número de ligações ativas:
$ c1=`cat server.log | grep CONNECT | wc -l`; c2=`cat server.log | grep CLOSE | wc -l`; let "c=c1-c2"; echo $c
0

# calcular o número total de pedidos processados pelo servidor:
$ cat server.log | grep REQUEST | wc -l
4

# obter a lista de clientes únicos:
$ cat server.log | awk '{print $2}' | sort | uniq
127.0.0.1:7654
127.0.0.1:7668

# contar o número de pedidos de cada cliente único:
$ for c in `cat server.log | awk '{print $2}' | sort | uniq`; do n=`cat server.log | grep $c | grep REQUEST | wc -l`; echo $c $n; done
127.0.0.1:7654 3
127.0.0.1:7668 1

# para cada cliente único, calcular o tempo durante o qual o cliente permaneceu ligado ao servidor:
$ for c in `cat server.log | awk '{print $2}' | sort | uniq`; do t1=`cat server.log | grep $c | grep CONNECT | awk '{print $1}'`; t2=`cat server.log | grep $c | grep CLOSE | awk '{print $1}'`; let "t=t2-t1"; echo $c $t; done
127.0.0.1:7654 59
127.0.0.1:7668 31

```

2.3 Gestão da Concorrência

No servidor vão existir três recursos partilhados que podem ser acedidos e modificados concorrentemente por múltiplas *threads*:

- o ficheiro de log (`server.log`), onde são registados os eventos;
- o número de clientes ativos, que deve refletir em tempo real quantos clientes estão ligados;
- a lista encadeada (*linked list*).

Assim, é necessário garantir que certas operações são realizadas de forma atómica. Por exemplo, ao atualizar a lista encadeada, a operação não deve ser interrompida por outra *thread* que aceda ou modifique a mesma estrutura, para evitar inconsistências ou corrupção de dados. É essencial identificar as secções críticas que envolvem estas variáveis e proteger o seu acesso através de mecanismos de sincronização. Para tal, podem ser utilizados *mutexes* (tipo `pthread_mutex_t`) e/ou variáveis condicionais (tipo `pthread_cond_t`). A forma concreta de implementação fica ao critério dos alunos, mas deve garantir que:

- não ocorrem escritas concorrentes no ficheiro de log;
- a atualização do número de clientes ativos é feita de forma segura;
- as operações sobre a lista encadeada são executadas em exclusão mútua, prevenindo acessos simultâneos que comprometam a consistência da estrutura.

3 Makefile

Os alunos deverão manter o `Makefile` usado na 2ª fase do projeto, atualizando-o para compilar novo código, se

necessário.

4 Entrega

A entrega da 3ª fase do projeto tem de ser feita de acordo com as seguintes regras:

1. Colocar todos os ficheiros do projeto, bem como o ficheiro `README` mencionado abaixo, num ficheiro com compressão no formato ZIP. O nome do ficheiro será **SD-XX-projeto3.zip** (XX é o número do grupo).
2. Submeter o ficheiro **SD-XX-projeto3.zip** na página da disciplina no moodle da FCUL, utilizando a atividade disponibilizada para tal. Apenas um dos elementos do grupo deve submeter, considerando-se apenas a submissão mais recente no caso de existirem várias.

O ficheiro ZIP deverá conter uma diretoria cujo nome é **SD-XX**, onde **XX** é o número do grupo. Nela serão colocados:

- o ficheiro **README**, onde os alunos podem incluir informações que julguem necessárias (e.g., limitações na implementação);
- diretorias adicionais, nomeadamente:
 - **include**: para armazenar os ficheiros `.h`;
 - **source**: para armazenar os ficheiros `.c`;
 - **lib**: para armazenar bibliotecas;
 - **object**: para armazenar os ficheiros objeto;
 - **binary**: para armazenar os ficheiros executáveis.
- um ficheiro **Makefile** que permita a correta compilação de todos os ficheiros entregues. Não devem ser incluídos no ficheiro ZIP os ficheiros objeto (`.o`) ou executáveis que são construídos pelo `Makefile`. Caso sejam usados os ficheiros objeto da 1ª fase do projeto disponibilizados aos grupos, estes **devem** ser incluídos no ficheiro ZIP.

Na entrega do trabalho, é ainda necessário ter em conta que:

- **Se não for incluído um Makefile, se o mesmo não compilar os ficheiros fonte, ou se houver erros de compilação (isto é, se não forem criados os ficheiros objeto e executáveis), o trabalho é considerado nulo.** Na página da disciplina, no Moodle, podem encontrar documentos do utilitário `make` e dos ficheiros `Makefile` (cortesia da disciplina de Sistemas Operativos).
- Todos os ficheiros entregues devem começar com um cabeçalho com três ou quatro linhas de comentários a dizer o número do grupo e o nome e número dos seus elementos.
- Os programas são testados no ambiente dos laboratórios de aulas, pelo que se recomenda que os alunos testem os seus programas neste mesmo ambiente.

O prazo de entrega é dia 16/11/2025, até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

5 Autoavaliação de contribuições

Cada aluno tem de preencher no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo para o projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário até a data limite de entrega do projeto **sofrem uma penalização na nota de 20%.**
- Caso existam assimetrias significativas entre as respostas de cada elemento do grupo, o grupo poderá ser chamado para as explicar.
- Se as contribuições individuais forem diferentes, isso será refletido na nota de cada elemento do grupo, levando à atribuição de notas individuais diferentes.

O prazo de preenchimento desde formulário é o mesmo que a entrega do projeto (16/11/2025, até às 23:59 horas).

6 Plágios

Não é permitido aos alunos partilharem códigos com soluções, ainda que parciais, de nenhuma parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão testados por um verificador de plágio. Caso alguma irregularidade seja encontrada, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto das plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) podem não resolver corretamente as alíneas descritas no projeto, e (3) incluem padrões característicos deste tipo de ferramenta,

as quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários. Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso 700. Se os ficheiros estiverem no GitHub, garantam que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.

7 Bibliografia

- [1] B. W. Kernighan, D. M. Ritchie, C Programming Language, 2nd Ed, Prentice-Hall, 1988.
- [2] Wikipedia. Linked List. https://en.wikipedia.org/wiki/Linked_list
- [3] <https://developers.google.com/protocol-buffers/docs/overview>